

Building the Data Model: From Raw Data to Analysis-Ready Tables

LS100 — Module 00A · Research Plans & Proposals

Last updated: 2026-09-30

Authored by **Souvik Mandal, Ph.D.**

Project Leader & Instructor, Computational Behavioral Sciences, LS100, FAS, Harvard University | LinkedIn ID: [souvik-mandal-phd](#)

A **data model** is the blueprint of how your data is organized at every step between the sensor and the statistical test. *Converting the Problem to a Research Framework* (Section 5) explains why you need one. This guide is the procedure for building it.

1. THE THREE STAGES OF DATA

Every LS100 project moves data through the same three stages:

Stage	What it is	One row is
Primary	What the sensor recorded (video, audio, logs). Never edited.	not a table — one file per recording
Secondary	What a model or algorithm extracts from one primary file.	one unit of extraction (a frame, a time window, an event)
Tertiary	Analysis-ready variables derived from the secondary data.	one unit of analysis

You will describe this flow in three passes, each more concrete than the last: **conceptual** (a diagram), **logical** (table schemas), and **physical** (folders, files, and Python).

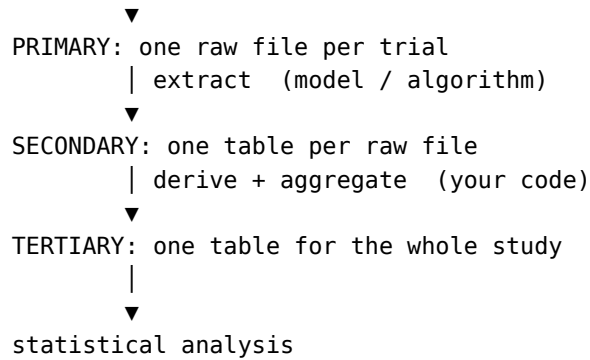
2. ASSIGNMENT 02: WHAT YOU HAND IN

One Jupyter notebook that contains the diagram, the schemas, and the folder layout in Markdown cells, followed by code that takes **one** primary file and produces its **secondary dataframe**. Producing the tertiary dataframe as well earns a bonus.

3. PASS 1 — CONCEPTUAL: DRAW THE FLOW

No technical detail yet. Name the **entities** your study tracks (participant, session, trial, recording) and draw how data moves between stages. Every arrow must be labelled with the transformation that performs it.

```
[Entity: participant → session → trial]
      | record
```



If you cannot name the transformation on an arrow, you have found a gap in your plan.

4. PASS 2 — LOGICAL: WRITE THE TABLE SCHEMAS

Write one schema for the secondary table and one for the tertiary table, independent of any software. Each column gets one line:

Column	Type	Unit	Role	Comes from
participant_id	text	—	ID	file name
<unit_index>	integer	—	ID	extraction
<feature>	float	<unit>	measure	<model or formula>

Three rules:

1. **IDs on every row.** Each table carries the identifiers of every entity above it, so a row can always be traced back to its participant, session, trial, and raw file.
2. **One row, one meaning.** State it in a sentence under each schema: “one row is one _____.”
3. **Every column has a source.** A tertiary column names the secondary columns and the formula that produce it. A column with no source cannot be computed.

Add a separate **metadata table** for attributes recorded by hand (one row per participant or session: condition, date, consented covariates). It joins to the tertiary table through the IDs.

5. PASS 3 — PHYSICAL: FOLDERS, FILE NAMES, AND PYTHON

Folders. One folder per stage; data only moves downward.

```

project/
├── data/
│   ├── 01_primary/      # raw files, read-only
│   ├── 02_secondary/    # one table per raw file
│   ├── 03_tertiary/     # one analysis-ready table
│   └── metadata.csv
└── notebooks/
  
```

File names. Encode the IDs in the name, in a fixed order, separated by underscores: <participant>_<session>_<trial>.<ext>. No spaces, no “final”, no dates typed by hand. A secondary file keeps the name of the primary file it came from, so the link between them never depends on memory.

Python skeleton. Declare the schema in code before writing any extraction logic, then make the notebook check its own output against it.

```
from pathlib import Path
import pandas as pd

RAW = Path("data/01_primary/P01_S01_T01.mp4")          # one primary
file
ids = dict(zip(["participant_id", "session_id", "trial_id"],
               RAW.stem.split("_")))                    # IDs from the
file name

SECONDARY_COLS = [*ids, "unit_index", "time_s", "feature_1"] # your
Pass 2 schema

def extract(raw_path: Path) -> list[dict]:
    """Primary → secondary: return one dict per unit of extraction."""
    # the extraction code goes here
    return []

secondary =
pd.DataFrame(extract(RAW)).assign(**ids).reindex(columns=SECONDARY_COLS)
assert secondary.columns.tolist() == SECONDARY_COLS

out = Path("data/02_secondary") / f"{RAW.stem}.csv"
out.parent.mkdir(parents=True, exist_ok=True)
secondary.to_csv(out, index=False)
```

For the extraction step, adapt the course notebooks: [pose estimation from one video](#) for video data, and [audio data foundation](#) for audio data.

Bonus — tertiary. Group the secondary dataframe by its IDs, compute the derived variables in your tertiary schema, and save the result to 03_tertiary/.

6. NOTEBOOK RULES

- Every code cell is preceded by a Markdown cell that states **why** the step is needed and **what** the code does. Write it for a classmate who has not seen your project.
- The notebook runs top to bottom on a fresh kernel, from one raw file, without manual edits between cells.
- Paths are relative to the project folder; nothing points to your personal machine.
- The last cell displays `secondary.head()` and `secondary.shape`, so the output can be checked against your schema at a glance.